WebSocket integration

Обновлено 18 апреля 2026, 19:23 Андрей Коновалов

1. How WebSocket Communication Works in Tournament Hub

The project follows the REST + WebSocket hybrid pattern defined in the requirements (API-7 , API-8 , WS-1 –WS-6)

- **REST** handles initial state load, mutations (submit, vote, join, start), and authentication.
- **WebSocket** handles real-time state propagation (clients receive server-pushed events without polling.)

The flow is:

1. Client authenticates via REST (JWT token).
2. Client opens a WebSocket connection, passing the JWT for identity.
3. Client subscribes to a tournament room (e.g., `tournament:{id}`).
4. The server pushes events whenever tournament state changes (phase transitions, submissions, votes, round completions, etc.).
5. Clients update their local UI state from the event payload — no follow-up REST call required (WS-4).

Since BR-16 and WS-6 explicitly state that WebSocket event history is not stored, the WebSocket layer is a pure delivery channel over the current domain state. State is authoritative in PostgreSQL; WebSocket is the notification pipe.

2. Recommended Library

The best option for the project is `@nestjs/websockets` + `@nestjs/platform-socket.io` .

Reasons:

1. **NestJS-native**: Gateways (`@WebSocketGateway`) integrate cleanly with NestJS DI, guards, interceptors, and pipes — no architectural friction.
2. **Room-based broadcasting**: Socket.IO 's room model maps perfectly to `tournament:{id}` subscriptions. Broadcasting to all tournament subscribers is a one-liner: `this.server.to(roomId).emit(event, payload)` .
3. **Fallback transports**: Socket.IO handles WebSocket with automatic fallback to long-polling — useful for environments where WebSockets may be blocked.
4. **JWT authentication in handshake**: Socket.IO handshake supports passing the token via `auth` object or query params, which integrates with existing JWT guards.
5. **Single-instance deployment**: NFR-10 and NFR-11 confirm one backend instance. Socket.IO without an adapter is sufficient.

Gateway Structure

A single `TournamentGateway` per tournament domain is sufficient for MVP. It handles:

- `handleConnection` / `handleDisconnect`
- `joinTournament` message (client subscribes to a room)
- Emitting all server-to-client events

Recommended internal wiring: `@nestjs/event-emitter` (lightweight internal event bus). Domain services emit internal events (e.g., `tournament.round.created`), and the gateway subscribes and re-emits over Socket.IO . This keeps gateways decoupled from business logic.

3. WebSocket API Documentation

The best option for documenting WebSocket events is `asyncapi` spec + `@asyncapi/cli`.
It is the industry standard for event-driven APIs.

Reasons:

1. **OpenAPI-equivalent for events**: AsyncAPI provides the same structured, machine-readable contract for WebSocket events that Swagger/OpenAPI provides for REST — satisfying API-5 in spirit and intent.
2. **Tooling**: `@asyncapi/cli` generates interactive HTML documentation from the spec, equivalent to the Swagger UI already used for REST. The team gets a single browsable API reference covering both REST and WebSocket.
3. **Frontend contract clarity**: A formal AsyncAPI spec gives frontend developers a precise payload contract per event — no ambiguity from free-form markdown.
4. **Event set is well-defined now**: With 10–11 events fully drafted in this research, writing the initial AsyncAPI spec is low effort and pays off immediately.
5. **Consistency**: The project already uses OpenAPI (Swagger) for REST documentation. Adopting AsyncAPI keeps the documentation approach consistent and tooling-aligned across both channels.

Notes: WebSocket events are documented in an AsyncAPI 3.0 spec file (`asyncapi.yaml` at project root). HTML docs are generated via `@asyncapi/cli`. The spec is maintained alongside the codebase and treated as the authoritative WebSocket contract between backend and frontend.

One more option for documentation is Manual markdown documentation (Structured event catalogue in a `.md` file).

Markdown has no tooling — nothing prevents it from drifting silently when payloads change. The project already uses Swagger for REST, so stopping at a text file for WebSocket creates two inconsistent documentation tiers. AsyncAPI fills that gap with the same generated, interactive, lintable contract.

4. Tournament WebSocket Events Draft

Each tournament has one room: `tournament:{tournamentId}`.
All events for that tournament are broadcast to this room.

For each event: **trigger**, **recipients**, **payload shape**, **full vs. incremental**.

`tournament:participant_joined`

Trigger: A user successfully joins a tournament (POST `/tournaments/:id/join` or owner auto-join on creation).
Recipients: All clients in `tournament:{id}` room.
Payload shape:

```
{
  "tournamentId": "uuid",
  "participant": {
    "userId": "uuid",
    "username": "string",
    "joinedAt": "ISO8601"
  },
  "participantCount": 3
}
```

Full vs. incremental: Incremental. Only the new participant is sent; client appends to its local list.
Notes: Relevant only in `draft` status. After tournament start, BR-8 freezes the participant list.

`tournament:participant_left`

Trigger: A participant leaves the tournament before it starts.
Recipients: All clients in room.
Payload shape:

```
{
  "tournamentId": "uuid",
  "userId": "uuid",
```

```
  "participantCount": 2
}
```

Full vs. incremental: Incremental.
Notes: QUESTIONABLE — no leave action is specified in FR or BR. BR-8 freezes participants after start. No endpoint for leaving.

`tournament:started`

Trigger: Owner calls POST `/tournaments/:id/start`. Tournament transitions from `draft` → `active`. First round is created and set to `submission` phase (FR-16).
Recipients: All clients in room.
Payload shape:

```
{
  "tournamentId": "uuid",
  "status": "active",
  "startedAt": "ISO8601",
  "firstRound": {
    "roundId": "uuid",
    "roundNumber": 1,
    "phase": "submission",
    "startedAt": "ISO8601"
  }
}
```

Full vs. incremental: Partial aggregate — tournament status + first round summary. Client does not need to re-fetch.

`round:created`

Trigger: System automatically creates the next round after the previous one completes (FR-36). Not emitted for the first round — `tournament:started` covers that case.

Recipients: All clients in room.
Payload shape:

```
{
  "tournamentId": "uuid",
  "round": {
    "roundId": "uuid",
    "roundNumber": 2,
    "phase": "submission",
    "startedAt": "ISO8601"
  }
}
```

Full vs. incremental: Incremental. Client adds the new round to its local round list.
Notes: Per BR-5 , only one active round at a time. This event signals that the new active round is in `submission` phase immediately.

`round:phase_changed`

Trigger: Submission phase ends → round transitions to `voting` phase (FR-19).
Recipients: All clients in room.
Payload shape:

```
{
  "tournamentId": "uuid",
  "roundId": "uuid",
  "roundNumber": 1,
  "previousPhase": "submission",
  "currentPhase": "voting",
  "transitionedAt": "ISO8601"
}
```

Full vs. incremental: Incremental — phase field update only.
Notes: This is the only mid-round phase transition. Submission → Voting is the single transition (BR-6). After voting ends, `round:completed` is used instead.

`submission:created`

Trigger: A participant submits their material during submission phase (FR-21).
Recipients: All clients in room.
Payload shape:

```
{
  "tournamentId": "uuid",
  "roundId": "uuid",
  "submission": {
    "submissionId": "uuid",
    "participantId": "uuid",
    "contentType": "string",
    "content": "string",
    "submittedAt": "ISO8601"
  }
}
```

Full vs. incremental: Incremental.

`submission:updated`

Trigger: A participant updates their submission before the submission phase ends (FR-24).
Recipients: All clients in room.
Payload shape:

```
{
  "tournamentId": "uuid",
  "roundId": "uuid",
  "submission": {
    "submissionId": "uuid",
    "participantId": "uuid",
    "contentType": "string",
    "content": "string",
```

```
    "updatedAt": "ISO8601"
  }
}
```

Full vs. incremental: Full submission object (replaces previous).

`vote:upserted`

Trigger: A user casts or changes their vote during voting phase (FR-26 , FR-28 , FR-31).
Recipients: All clients in room.
Payload shape:

```
{
  "tournamentId": "uuid",
  "roundId": "uuid",
  "submissionId": "uuid",
  "vote": {
    "userId": "uuid",
    "value": "for | against",
    "votedAt": "ISO8601"
  },
  "submissionVoteSummary": {
    "forCount": 5,
    "againstCount": 2
  }
}
```

Full vs. incremental: Incremental for vote, includes running tally for the submission.
Notes: A single `vote:upserted` event covers both create and update cases. Including the running tally (`submissionVoteSummary`) satisfies WS-4 (no re-fetch needed) and FR-31 (update current results).

`round:completed`

Trigger: Voting phase ends. System finalizes round results, recalculates cumulative scores, updates time statistics (FR-32 , FR-33 , FR-34 , FR-35).
Recipients: All clients in room.
Payload shape:

```
{
  "tournamentId": "uuid",
  "round": {
    "roundId": "uuid",
    "roundNumber": 1,
    "status": "completed",
    "completedAt": "ISO8601",
    "results": [
      {
        "submissionId": "uuid",
        "participantId": "uuid",
        "forVotes": 5,
        "againstVotes": 2,
        "score": 3
      }
    ]
  },
  "leaderboard": [
    {
      "participantId": "uuid",
      "username": "string",
      "cumulativeScore": 10,
      "totalTime": 12345
    }
  ]
}
```

Full vs. incremental: Full round result + updated leaderboard. This is a state snapshot for the completed round.
Notes: Satisfies FR-32 (emit round completion event), FR-33 (cumulative scores), FR-35 (aggregate time). The leaderboard in the payload allows the client to render updated standings without a REST call.

`tournament:finished`

Trigger: Last round completes. System determines the winner and sets tournament status to `finished` (FR-37 , FR-38 , FR-39 , FR-40 , FR-41).
Recipients: All clients in room.
Payload shape:

```
{
  "tournamentId": "uuid",
  "status": "finished",
  "finishedAt": "ISO8601",
  "winner": {
    "participantId": "uuid",
    "userId": "uuid",
    "username": "string",
    "cumulativeScore": 25,
    "totalTime": 34567,
    "connectedAt": "ISO8601"
  },
  "finalLeaderboard": [
    {
      "rank": 1,
      "participantId": "uuid",
      "username": "string",
      "cumulativeScore": 25,
      "totalTime": 34567
    }
  ]
}
```

Full vs. incremental: Full final state. This is the terminal event for a tournament room.
Notes: After this event, the server can remove the tournament room. Clients can disconnect from the room or keep listening for the final state display.

`tournament:state_sync`

Trigger: Client connects to (or reconnects to) a tournament room. Server emits current state proactively on join.
Recipients: The requesting client only (unicast, not broadcast).
Payload shape: Full aggregate tournament state — equivalent to the response of `GET /tournaments/:id/full`.
Full vs. incremental: Full aggregate.
Notes: This event serves the reconnection scenario. Since WS-6 states that event history is not stored, a reconnecting client cannot replay missed events — it receives the current aggregate state and re-renders from there. This aligns with API-7 ("REST for initial load") but provides the same data over WebSocket for the reconnection case, avoiding an extra HTTP round trip.

Questions:

1. **Submission visibility during submission phase**: Should all clients see submissions as they are created, or only after the submission phase ends?
2. **Vote anonymity**: Should `vote:upserted` include the voter's `userId`, or only the aggregate tally?
3. `tournament:state_sync` **vs REST on reconnect**: Should reconnection use `GET /tournaments/:id/full` (REST) or a dedicated `tournament:state_sync` WebSocket event?
4. `tournament:participant_left`: Is there a "leave tournament" action before start?
5. **Phase transition trigger**: Are phase transitions (submission - voting, voting - complete) triggered manually by the owner, by a timer, or by all participants completing their action? This affects how `round:phase_changed` and `round:completed` are triggered server-side.